OxigenScript
Intérprete Educativo

Manual de Usuario

Organización de Lenguajes y Compiladores I

Guía completa del intérprete OxigenScript:
instalación, sintaxis, ejemplos y reportes (AST, tokens y errores).

Autor: Angel Raúl Herrera Chilel
Carnet: 202402587
Sección: B
Fecha: 28 de agosto de 2026

Índice

Introducción **3**

1 Requerimientos e Instalación **4**
1.1 Crear el entorno e instalar dependencias . . . 4
1.2 Iniciar la aplicación . . . 4

2 Inicio del Programa **4**
2.1 Descripción de la interfaz . . . 5
2.2 Atajos de teclado . . . 5

3 Primer Programa **5**
3.1 Escribir y ejecutar . . . 5
3.2 Estructura general de un programa . . . 5

4 Manual del Lenguaje **6**
4.1 Variables y tipos . . . 6
4.1.1 Sintaxis . . . 6
4.1.2 Tipos de dato disponibles . . . 6
4.1.3 Valores por defecto . . . 6
4.1.4 Ejemplo . . . 6
4.1.5 Operadores de asignación . . . 7
4.2 Operadores . . . 7
4.2.1 Aritméticos . . . 7
4.2.2 Relacionales . . . 7
4.2.3 Lógicos . . . 8
4.3 Control de flujo . . . 8
4.3.1 Condicionales . . . 8
4.3.2 Bucles . . . 8
4.3.3 Bucles con etiquetas . . . 9
4.3.4 Match . . . 9
4.4 Funciones . . . 9
4.5 Structs . . . 10
4.6 Arreglos y strings . . . 10
4.7 Funciones embebidas . . . 11
4.8 Comentarios . . . 11

5 Páginas de Reportes **12**
5.1 Editor . . . 12
5.2 Errores . . . 12
5.3 Símbolos . . . 12
5.4 AST . . . 12

6 Interpretación de la Consola **12**

7 Guardado y Carga de Código **13**

8 Ejemplo Completo de Uso **13**
8.1 Resultado esperado en la consola . . . 14

9 Solución de Problemas **15**

10 Conclusiones y Recomendaciones **15**
10.1 Conclusiones 15
10.2 Recomendaciones 15

11 Anexos — Capturas de Pantalla **16**

Introducción

OxigenScript es un lenguaje de programación educativo con sintaxis inspirada en Rust. A diferencia de un compilador convencional, OxigenScript es un **intérprete**: analiza el código fuente, construye un árbol de sintaxis abstracta (AST) y lo ejecuta directamente.

La herramienta se compone de un conjunto de librerías escritas en Python que implementan el lenguaje y de un **entorno de desarrollo integrado (IDE) web** que permite escribir, ejecutar y revisar el resultado de cada programa, junto con sus reportes de tokens, errores y el grafo del AST.

Este manual está dirigido a los usuarios del sistema. No se requieren conocimientos previos de compiladores: basta conocer la sintaxis del lenguaje descrita en este documento y saber operar la interfaz web del editor.

Requerimientos e Instalación

Para ejecutar OxigenScript es necesario tener instalados los siguientes programas:

Requerimiento	Descripción
Python 3.14	Versión mínima requerida por el proyecto. Verificar con `python3 -version`.
uv	Gestor de entornos y dependencias de Python. Se usa para crear el entorno virtual.
Node.js	Necesario para el entorno de desarrollo web (Astro). Verificar con `node -version`.

Crear el entorno e instalar dependencias

1. Descomprima o clone el proyecto OxigenScript.
2. Dentro del directorio del proyecto, cree el entorno virtual e instale las dependencias de Python:

```
uv sync
```

3. Instale las dependencias del frontend web:

```
cd frontend
npm install
cd ..
```

Iniciar la aplicación

El proyecto incluye un script que levanta simultáneamente el backend (API) y el frontend (IDE web):

```
./start.sh
```

Al terminar, se mostrarán las direcciones de acceso:

```
Servidores listos
  Frontend: http://localhost:4321
  Backend:  http://localhost:8000
```

Para detener ambos servidores presione `Ctrl+C` en la terminal donde se ejecutó el `./start.sh`.

Inicio del Programa

Abra su navegador y diríjase a `http://localhost:4321`. La aplicación lo redirigirá automáticamente a la página del **Editor**.

Descripción de la interfaz

Área	Función
Barra de herramientas	Botones: Nuevo, Cargar, Guardar y Ejecutar, más el selector de tema (Claro/Oscuro).
Editor (centro)	Área de escritura de código OxigenScript con resaltado de sintaxis, numeración de líneas y autocompletado de corchetes.
Consola (inferior)	Panel colapsable que muestra la salida del programa y los avisos/errores.
Navegación (superior)	Enlaces: Editor, Errores, Símbolos y AST.

Atajos de teclado

El editor web (CodeMirror) incluye los atajos habituales de los editores:

Atajo	Acción
`Ctrl + S`	Guardar el código como archivo `.ox`
`Ctrl + Z` / `Ctrl + Y`	Deshacer / Rehacer
`Ctrl + F`	Buscar en el editor

Primer Programa

Escribir y ejecutar

1. En el editor, escriba el clásico `Hola Mundo`:

Listing 1: Primer programa

```
fn main() {
    println!("Hola OxigenScript!");
}
```

2. Haga clic en el botón **Ejecutar** (verde).

3. La consola inferior se desplegará y mostrará la salida:

```
Hola OxigenScript!
```

4. El botón **AST** en la navegación le mostrará el árbol de sintaxis abstracta del programa que acaba de ejecutar.

Estructura general de un programa

Un programa OxigenScript está formado por un conjunto de **funciones** y, opcionalmente, **structs**. Todo programa debe definir la función `main`, que es el punto de entrada:

1. Definir `structs` (opcional).

2. Definir `fns` auxiliares.

3. Definir `fn main()` y desde aquí invocar el resto.

Manual del Lenguaje

Variables y tipos

OxigenScript es de tipado estático. Las variables se declaran con `let` (son inmutables por defecto) y con `let mut` si se desea poder reasignarlas.

Sintaxis

```
let nombre = valor;                 // tipo inferido
let nombre: tipo = valor;           // tipo explícito
let nombre: tipo;                   // tipo explícito sin inicializador (valor por defecto)
let mut nombre = valor;             // variable mutable
let mut nombre: tipo = valor;
```

Tipos de dato disponibles

Tipo	Descripción
`i32`	Número entero (ej. `10`)
`f64`	Número decimal (ej. `3.14`)
`bool`	Valor booleano: `true` o `false`
`char`	Carácter (ej. `'A'`)
`String`	Cadena de texto (ej. `String::from("Hola")`)
`[T; N]`	Arreglo de tamaño fijo (ej. `[i32; 5]`)

Valores por defecto

Si se declara una variable con tipo pero sin inicializador, se le asigna un valor por defecto:

Tipo	Valor por defecto
`i32`	`0`
`f64`	`0.0`
`bool`	`false`
`String`	(cadena vacía)
`char`	`'\0'`

Ejemplo

Listing 2: Variables y shadowing

```
fn main() {
    let entero: i32 = 10;
    let decimal: f64 = 3.14;
    let activo: bool = true;
    let contador: i32;          // vale 0
    let mut suma = 0;
    suma = 5;

    // Shadowing: redeclarar una variable con el mismo nombre
```

```
    let x: i32 = 10;
    let x: f64 = 3.14;

    println!("{}", entero);
    println!("{}", contador);
    println!("{}", x);
}
```

Operadores de asignación

```
let mut x: i32 = 10;
x = 5;       // asignación simple
x += 3;      // suma compuesta
x -= 2;
x *= 4;
x /= 6;
x %= 5;
```

Operadores

Aritméticos

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación / repetición de String
/	División
%	Módulo
-x	Negación unaria

Listing 3: Uso de operadores aritméticos

```
let a: i32 = 10;
let b: f64 = 2.5;
let suma = a + b;                    // 12.5 (promoción a f64)
let producto = a * 5;                // 50
let division = 20 / 4;               // 5
let modulo = 17 % 5;                 // 2
let negativo = -a;                   // -10
let texto = String::from("Hola");
let repetido = texto * 3;            // "HolaHolaHola"
```

Relacionales

Operador	Descripción
==	Igual
!=	Diferente
> / <	Mayor / Menor
>= / <=	Mayor o igual / Menor o igual

Lógicos

Operador	Descripción
`!`	Negación (NOT)
`&&`	Y lógico (AND)
`||`	O lógico (OR)

Control de flujo

Condicionales

Listing 4: if / else if / else

```
fn main() {
    let x: i32 = 10;
    if x > 5 {
        println!("Mayor que 5");
    }

    let flag: bool = false;
    if flag {
        println!("Verdadero");
    } else if flag == false {
        println!("Falso");
    } else {
        println!("Otro caso");
    }
}
```

Bucles

Listing 5: while, loop y for

```
fn main() {
    // while
    let mut contador: i32 = 0;
    while contador < 3 {
        println!("{}", contador);
        contador = contador + 1;
    }

    // loop (infinito) con break
    let mut i: i32 = 0;
    loop {
        if i >= 3 { break; }
        println!("Loop {}", i);
        i = i + 1;
    }

    // for sobre un rango
    for j in 0..3 {
        println!("{}", j);
    }
```

```
}
```

Bucles con etiquetas

OxigenScript permite etiquetar bucles con `'etiqueta` y salir de uno anidado con `break 'etiqueta`:

Listing 6: Bucles etiquetados

```
fn main() {
    'outer: loop {
        'inner: loop {
            break 'outer;          // sale del bucle externo
            println!("Esto no se ejecuta");
        }
    }
    println!("Fuera de los loops");
}
```

Match

Listing 7: Estructura match

```
fn main() {
    let numero: i32 = 2;
    match numero {
        1 => println!("UNO"),
        2 => println!("DOS"),
        _ => println!("OTRO"),
    }
}
```

Funciones

Listing 8: Declaración y llamada de funciones

```
fn saludar(nombre: String) {
    println!("Hola, {}", nombre);
}

fn sumar(a: i32, b: i32) -> i32 {
    return a + b;
}

fn main() {
    saludar(String::from("Ana"));

    let resultado = sumar(10, 5);
    println!("Resultado: {}", resultado);
}
```

Las funciones devuelven valor con `return expresion;`. Si no declaran tipo de retorno (`->tipo`), son procedimientos (void).

Structs

Listing 9: Definición e instanciación de structs

```
struct Point {
    x: i32,
    y: i32,
}

struct Rectangle {
    position: Point,
}

fn main() {
    let punto = Point { x: 10, y: 20 };
    println!("{}", punto.x);           // 10

    // Struct anidado
    let rectangulo = Rectangle {
        position: Point { x: 5, y: 15 },
    };
    println!("{}", rectangulo.position.x);    // 5
}
```

Arreglos y strings

Listing 10: Arreglos

```
fn main() {
    let numeros: [i32; 5] = [10, 20, 30, 40, 50];
    println!("{}", numeros[0]);         // 10
    println!("{}", numeros.len());      // 5

    // Slice
    let parte = &numeros[1..4];
    println!("{:?}", parte);            // [20, 30, 40]

    // Acceso con arreglo
    let mut i = 0;
    while i < numeros.len() {
        println!("{}", numeros[i]);
        i = i + 1;
    }
}
```

Listing 11: Strings

```
fn main() {
    let saludo = String::from("Hola Mundo");
    println!("{}", saludo.len());               // 10
```

```
    println!("{}", saludo.contains("Mundo"));    // true
    println!("{}", saludo.replace("Mundo", "Rust")); // Hola Rust
    println!("{}", saludo.to_uppercase());       // HOLA MUNDO
    println!("{}", saludo.to_lowercase());       // hola mundo

    // Split devuelve un arreglo
    let palabras = saludo.split(" ");
    println!("{:?}", palabras);                  // ["Hola", "Mundo"]
}
```

Las **cadenas crudas** (raw strings) permiten escribir texto sin procesar secuencias de escape: `rÇ:\Users"` o `r"..."`.

Funciones embebidas

Función	Descripción
`println!("...", args)`	Imprime con formato `{}` o `{:?}`
`typeof(exp)`	Devuelve el tipo de la expresión como texto
`random(a, b)`	Número entero aleatorio entre `a` y `b`
`exp.len()`	Longitud de un string o arreglo
`exp.contains(s)`	¿Contiene la subcadena `s`?
`exp.replace(a, b)`	Reemplaza `a` por `b`
`exp.split(s)`	Divide el string y devuelve un arreglo
`exp.to_uppercase()`	Convierte a mayúsculas
`exp.to_lowercase()`	Convierte a minúsculas
`exp.reverse()`	Invierte un string o arreglo (muta la variable)
`exp.push_str(s)`	Concatena y actualiza el string (sentencia)

Listing 12: Ejemplos de funciones embebidas

```
fn main() {
    let texto = String::from("Hola Mundo");
    println!("{}", typeof(texto));          // String
    println!("{}", random(1, 10));          // aleatorio entre 1 y 10

    let mut numeros = [10, 20, 30];
    numeros.reverse();
    println!("{:?}", numeros);              // [30, 20, 10]
}
```

Comentarios

OxigenScript soporta comentarios de una línea y de bloque, que son ignorados por completo por el compilador.

Listing 13: Tipos de comentarios

```
// Este es un comentario de una sola línea

/* Este es un comentario
```

```
    de bloque multilínea */

fn main() {
    let x: i32 = 10;    // comentario al final de línea
}
```

Páginas de Reportes

La aplicación web dispone de cuatro páginas accesibles desde la navegación superior. Todas se actualizan al volver a ejecutar el programa en el Editor.

Editor

La página principal (`/dashboard`). Contiene el editor de código, la barra de herramientas y la consola de salida. Es donde se escribe y ejecuta el código.

Errores

La página `/errores` muestra la tabla de errores de la última ejecución, con su ubicación (línea y columna) y el mensaje. Los errores se distinguen por color según su categoría:

Color	Categoría	Descripción
Ámbar	Léxico	Carácter inválido en el código
Rojo	Sintáctico	Violación gramatical
Morado	Semántico	Error de tipos, scopes, etc.
Rosa	Ejecución	Error en tiempo de ejecución

Ejemplo de mensaje:

```
3:9 [Semantico] Se esperaba 'bool', pero se obtuvo 'i32'
```

Símbolos

La página `/simbolos` muestra la **tabla de tokens** reconocidos por el analizador léxico en la última ejecución, con el tipo de token, su lexema y la posición (línea y columna).

AST

La página `/ast` muestra el **grafo del Árbol de Sintaxis Abstracta** generado al ejecutar el último programa. Es una representación visual en PNG de cómo se estructuró el código en nodos.

> Los reportes de Errores, Símbolos y AST se generan con la última **ejecución**. Después de modificar el código, vuelva a pulsar **Ejecutar** y luego navegue a la página pertinente para ver datos actualizados.

Interpretación de la Consola

La consola, situada debajo del editor, muestra el resultado de cada ejecución. Es un panel colapsable (botón con flecha /).

Estado	Descripción
Verde	Salida normal del programa (`println!`).
Ámbar	Advertencias.
Rojo	Errores detectados.
Insignia	`OK`, `N advertencia(s)` o `N errores`.

Si el programa genera salida, esta se muestra junto con las advertencias. Solo si **no** hay salida **y** hay errores se indica que la ejecución no pudo llevarse a cabo.

Guardado y Carga de Código

Botón	Función
Nuevo	Limpia el editor y reinicia a un programa vacío.
Cargar	Abre un archivo `.ox` desde el disco y lo carga en el editor.
Guardar	Descarga el código actual como archivo `.ox`.

Además, el editor guarda automáticamente el código en el `localStorage` del navegador, por lo que el contenido se conserva al recargar la página.

Ejemplo Completo de Uso

Listing 14: Programa OxigenScript completo

```
struct Punto {
    x: i32,
    y: i32,
}

fn sumar(a: i32, b: i32) -> i32 {
    return a + b;
}

fn main() {
    println!("=== INICIO ===");

    // Variables y aritmética
    let a: i32 = 10;
    let b: f64 = 2.5;
    println!("{}", a + b);           // 12.5

    // Control de flujo
    let mut i: i32 = 0;
    while i < 3 {
        println!("{}", i);
        i = i + 1;
    }

    // Bucle etiquetado
    'outer: loop {
        'inner: loop {
            break 'outer;
        }
```

```
    }
    println!("Fuera de los loops");

    // Servei
    let punto = Punto { x: 10, y: 20 };
    println!("{}", punto.x);          // 10

    // Funciones
    let r = sumar(10, 5);
    println!("Resultado: {}", r);    // Resultado: 15

    // Arreglos
    let numeros = [10, 20, 30];
    let mut n = 0;
    while n < numeros.len() {
        println!("{}", numeros[n]);
        n = n + 1;
    }

    println!("=== FIN ===");
}
```

Resultado esperado en la consola

```
=== INICIO ===
12.5
0
1
2
Fuera de los loops
10
Resultado: 15
10
20
30
=== FIN ===
```

Solución de Problemas

Problema	Solución
El frontend no abre	Verifique que `./start.sh` reportó ambos servidores; abra `http://localhost:4321`.
El backend no inició	Revise el log en `/tmp/fastapi.log`.
Puerto ocupado	Compruebe que nada más usa los puertos 8000/4321; ciérrelo o cambie el puerto en `start.sh`.
La consola está vacía	Ejecute de nuevo el programa; si hay errores, se mostrarán en rojo en la consola y en la página Errores.
Los reportes no cambian	Re-ejecute el código y vuelva a la página; los datos se actualizan con la última ejecución.
Sintaxis no reconocida	Consulte las secciones de este manual; recuerde terminar las sentencias con `;`.

Conclusiones y Recomendaciones

Conclusiones

- OxigenScript permite desarrollar y ejecutar programas con sintaxis inspirada en Rust mediante un intérprete Python.
- La interfaz web integra editor, ejecución y reportes (tokens, errores y AST) en un único flujo de trabajo.
- El reporte de errores con ubicación exacta (línea y columna) facilita la identificación y corrección de problemas.

Recomendaciones

- Siempre defina la función `main()` como punto de entrada.
- Use `let mut` solo cuando deba reasignar una variable.
- Revise la consola y la página de Errores cuando la salida esté vacía u obtenga resultados inesperados.
- Guarde sus programas como archivos `.ox` con el botón **Guardar** para poder reutilizarlos.

Anexos — Capturas de Pantalla

En esta sección se presentan las capturas de pantalla de la herramienta.

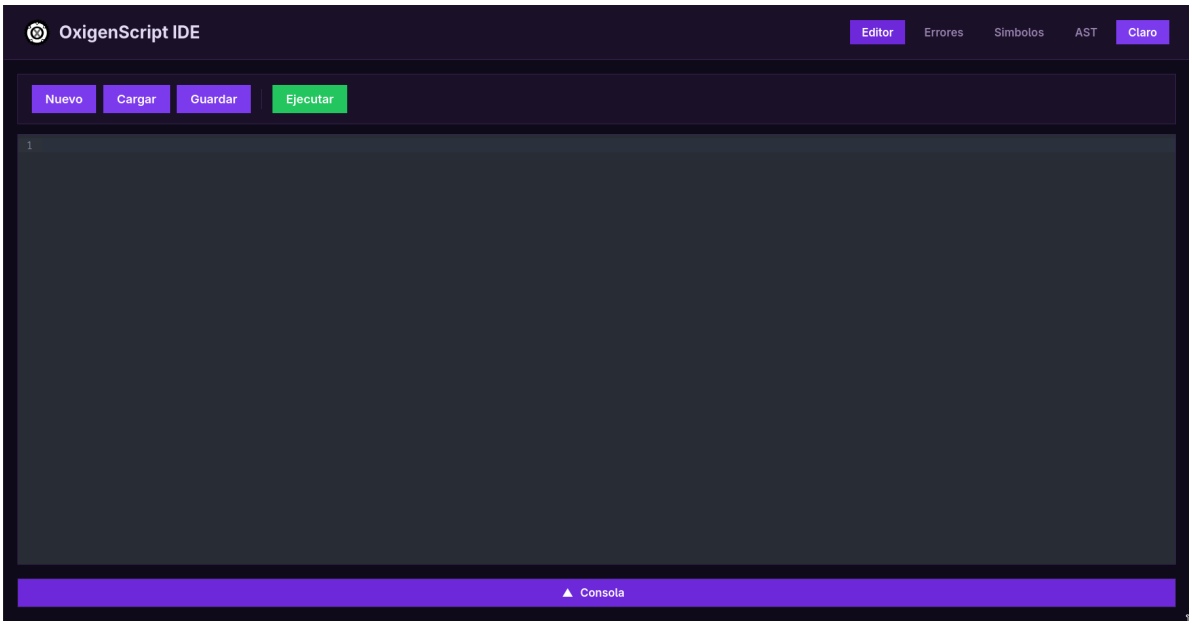


Figura 1: Ventana principal — Editor OxigenScript

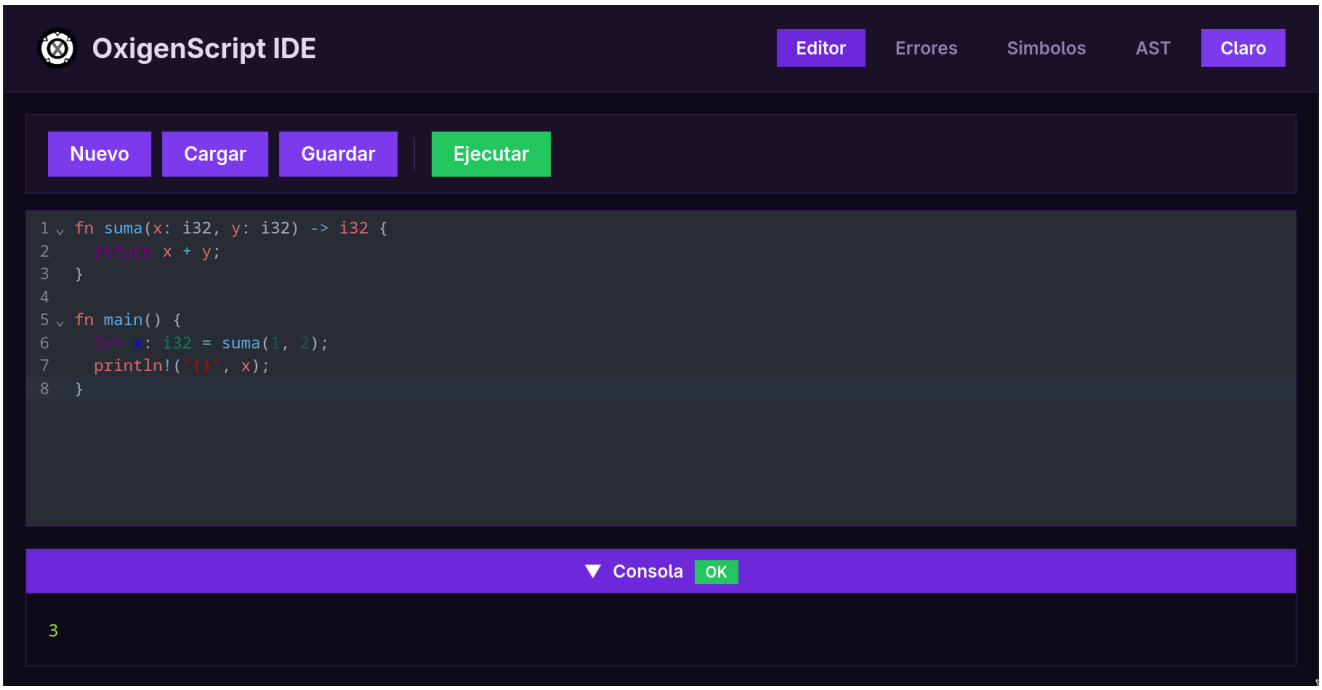


Figura 2: Consola con la salida de un programa

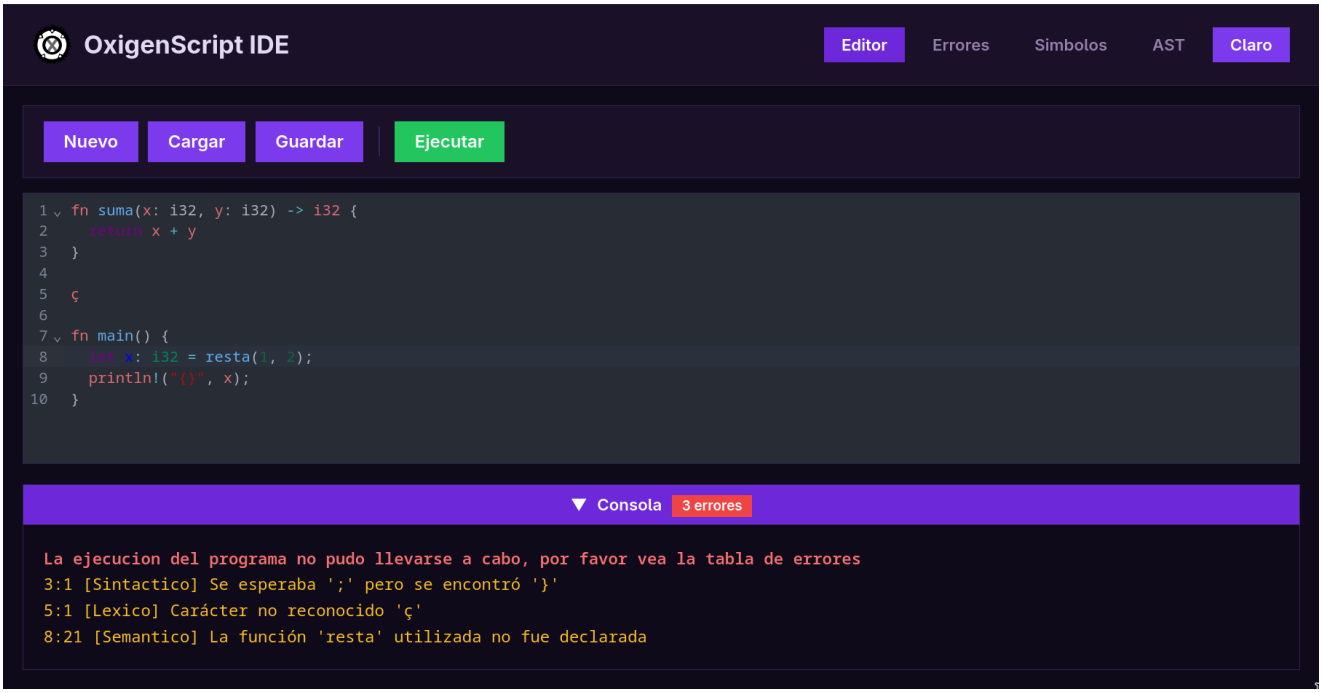


Figura 3: Despliegue de errores en consola

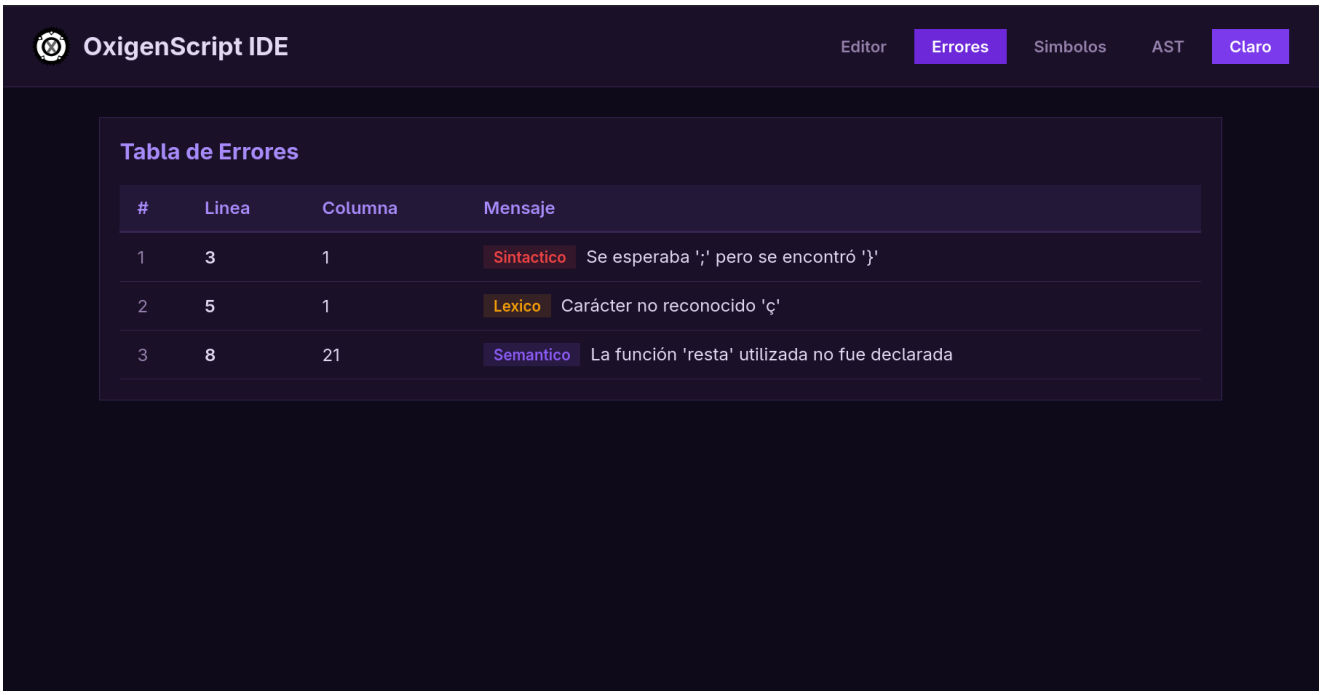
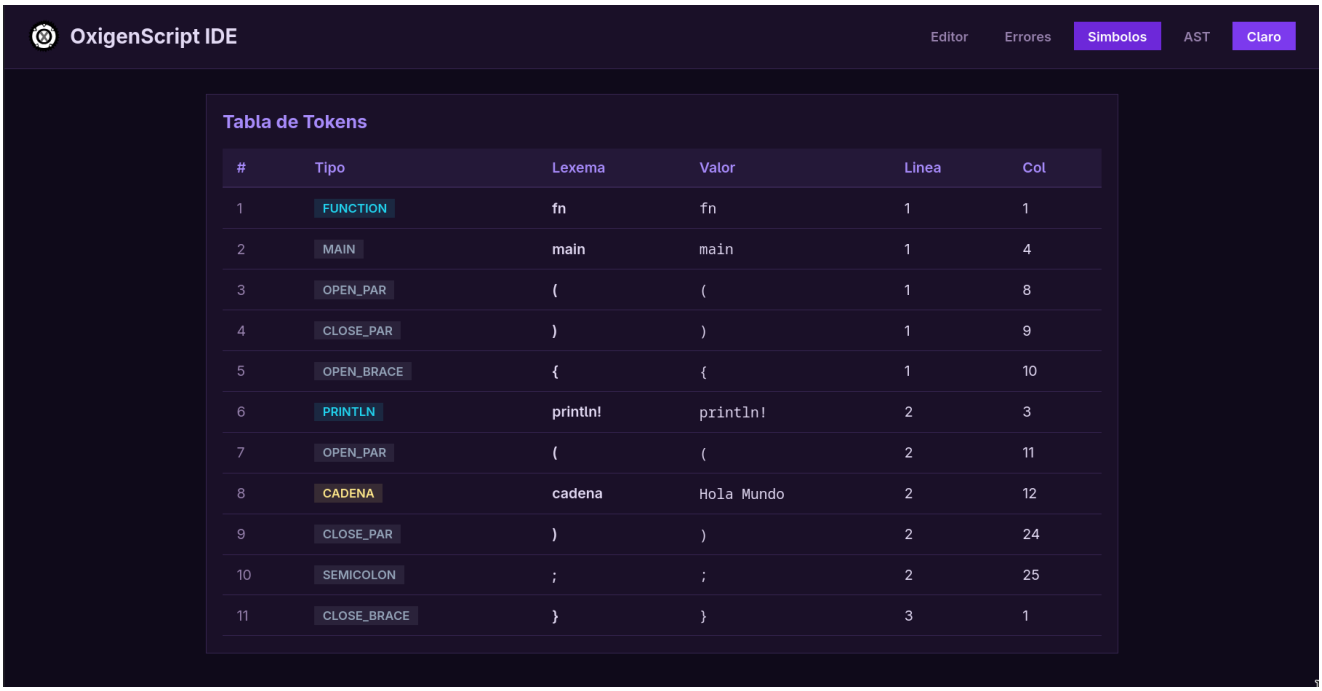


Figura 4: Página de Errores



#	Tipo	Lexema	Valor	Linea	Col
1	FUNCTION	fn	fn	1	1
2	MAIN	main	main	1	4
3	OPEN_PAR	(	(	1	8
4	CLOSE_PAR	)	)	1	9
5	OPEN_BRACE	{	{	1	10
6	PRINTLN	println!	println!	2	3
7	OPEN_PAR	(	(	2	11
8	CADENA	cadena	Hola Mundo	2	12
9	CLOSE_PAR	)	)	2	24
10	SEMICOLON	;	;	2	25
11	CLOSE_BRACE	}	}	3	1

Figura 5: Tabla de Símbolos

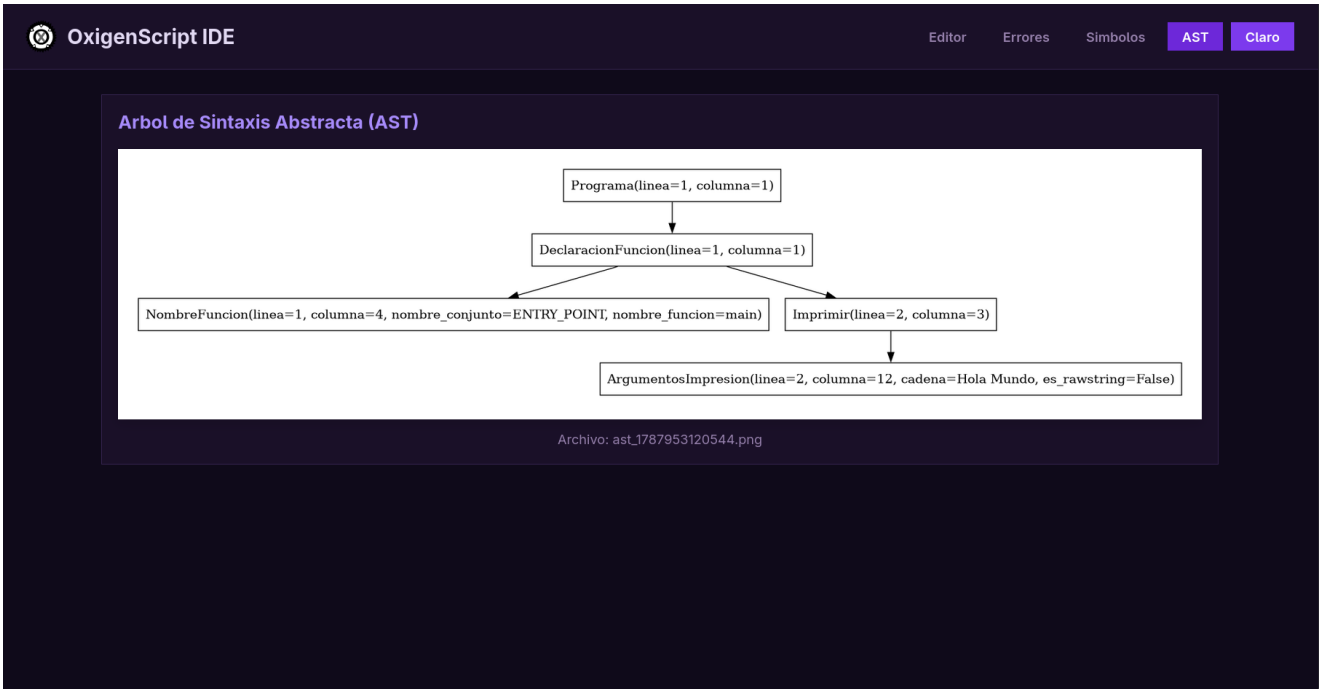


Figura 6: Página del Árbol de Sintaxis Abstracta (AST)